

Fortgeschrittene Lineare Algebra und Analytische Geometrie

06 Hyperebenen, Abstand und Margin Classifier

Martin Zaefferer

Quellen

- Deisenroth, Faisal, Ong. 2021. Mathematics for Machine Learning. Cambridge University Press.
- James, Witten, Hastie, Tibshirani. 2023. An Introduction to Statistical Learning – with Applications in R. Springer.

Erinnerung

Koordinatengleichung einer Ebene

- Parameterdarstellung

$$\vec{x} = \vec{p} + \lambda \cdot \vec{q} + \mu \cdot \vec{r}$$

- Koordinatengleichung:

$$n_0 + n_1x_1 + n_2x_2 + n_3x_3 = 0$$

- mit:

$$\vec{n} = \vec{q} \times \vec{r}$$

- und:

$$n_1p_1 + n_2p_2 + n_3p_3 = -n_0$$

Abstand Punkt Ebene

- Gegeben: Punkt $\vec{p}$, Ebene E in Koordinatenform $n_0 + n_1x_1 + n_2x_2 + n_3x_3 = 0$
- Gesucht: Abstand der beiden
- Lösung:

$$\frac{|\vec{p}^T \vec{n} + n_0|}{|\vec{n}|}$$

Hyperebenen

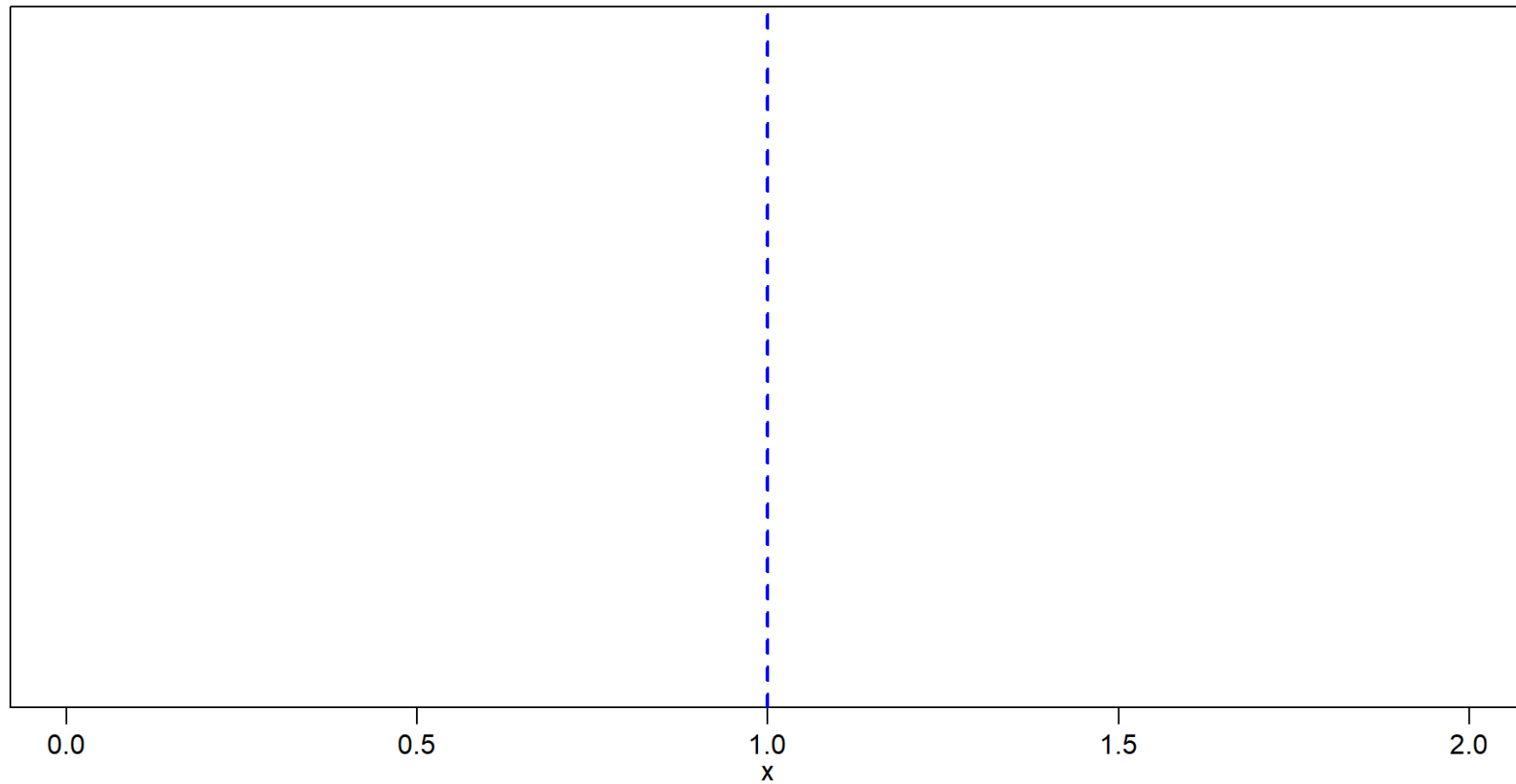
Was ist eine Hyperebene

- Definition: **Hyperebene** in $\mathbb{R}^d$ ist ein *affiner Unterraum* mit Dimension $d - 1$
- Jeder Punkt $\vec{x}$ auf Ebene erfüllt $n_0 + n_1x_1 + n_2x_2 + \dots + n_dx_d = 0$
- Oder auch:

$$n_0 + \vec{n}^T \vec{x} = 0$$

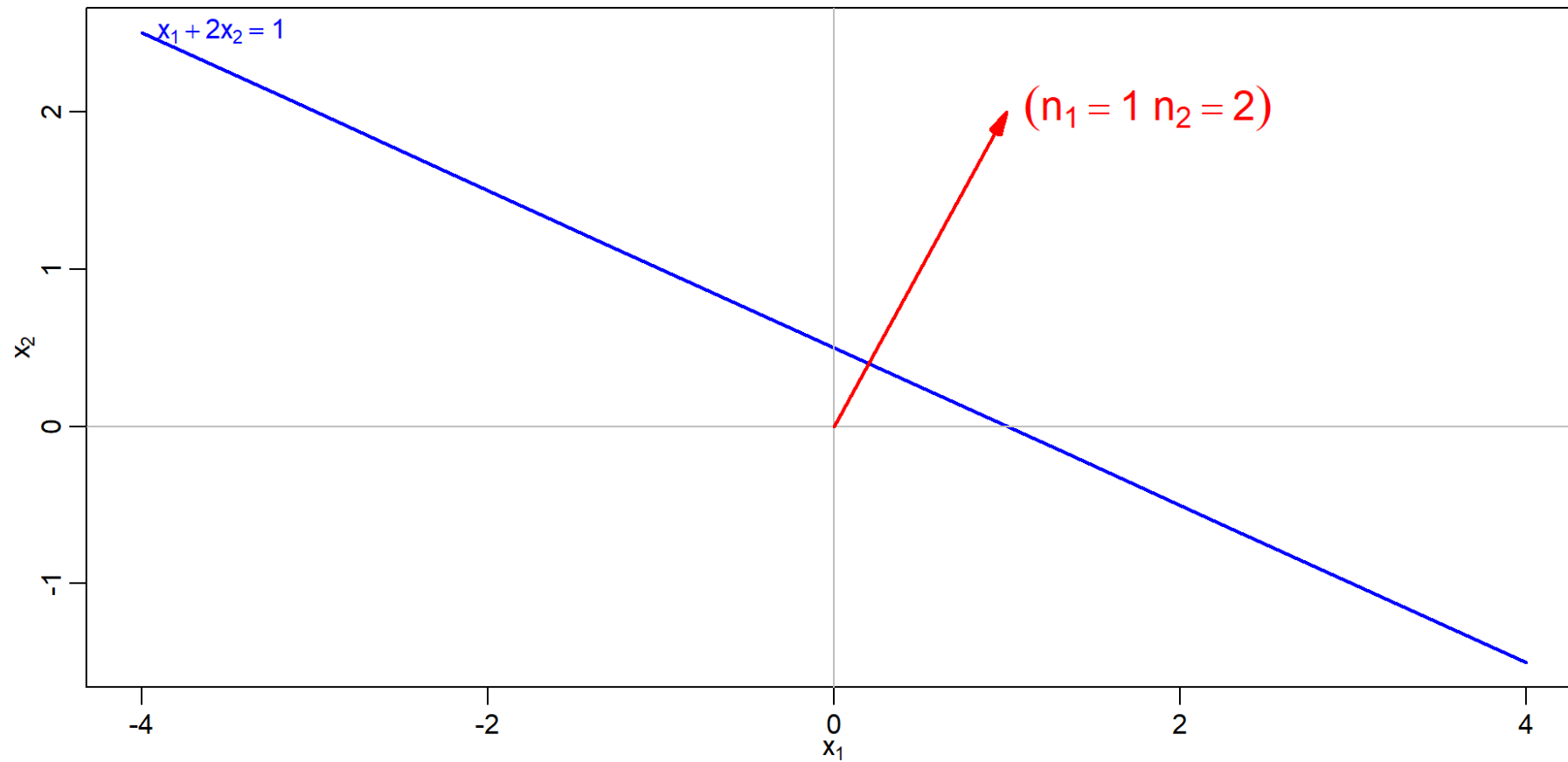
- 1 Dimension: Hyperebene ist Punkt
- 2 Dimensionen: Gerade
- 3 Dimensionen: Ebene
- Wenn $n_0 = 0$: Hyperebene verläuft durch Ursprung (somit Untervektorraum), sonst nicht.
- $\vec{n} = (n_1 \quad n_2 \quad \dots \quad n_d)$ ist der 'Normalenvektor', ist orthogonal zur Hyperebene

Hyperebene in 1D

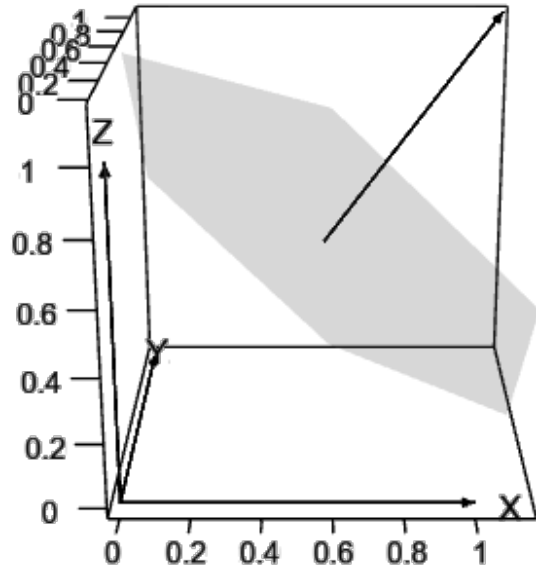


Hier: $n_0 + n_1x = 0$, mit $n = 1$ und $n_0 = -1$

Hyperebene in 2D



Hyperebene in 3D



Hier: $\vec{n}^T \vec{x} + n_0$ mit $\vec{n} = \begin{pmatrix} 1/\sqrt{3} \\ 1/\sqrt{3} \\ 1/\sqrt{3} \end{pmatrix}$ und $n_0 = -1$

Abstand

Abstand

Abstand eines Punktes $\vec{p}$ zur Ebene $E : \vec{n}^T \vec{x} + n_0$

$$d = \frac{|\vec{p}^T \vec{n} + n_0|}{|\vec{n}|}$$

Oder wenn $\vec{n}$ die Länge 1 hat (hierfür muss auch n_0 angepasst werden):

$$d = |\vec{p}^T \vec{n} + n_0|$$

Frage: was bedeutet hier der Absolutwert? Warum/wann kann der zugrundeliegende Wert negativ werden?

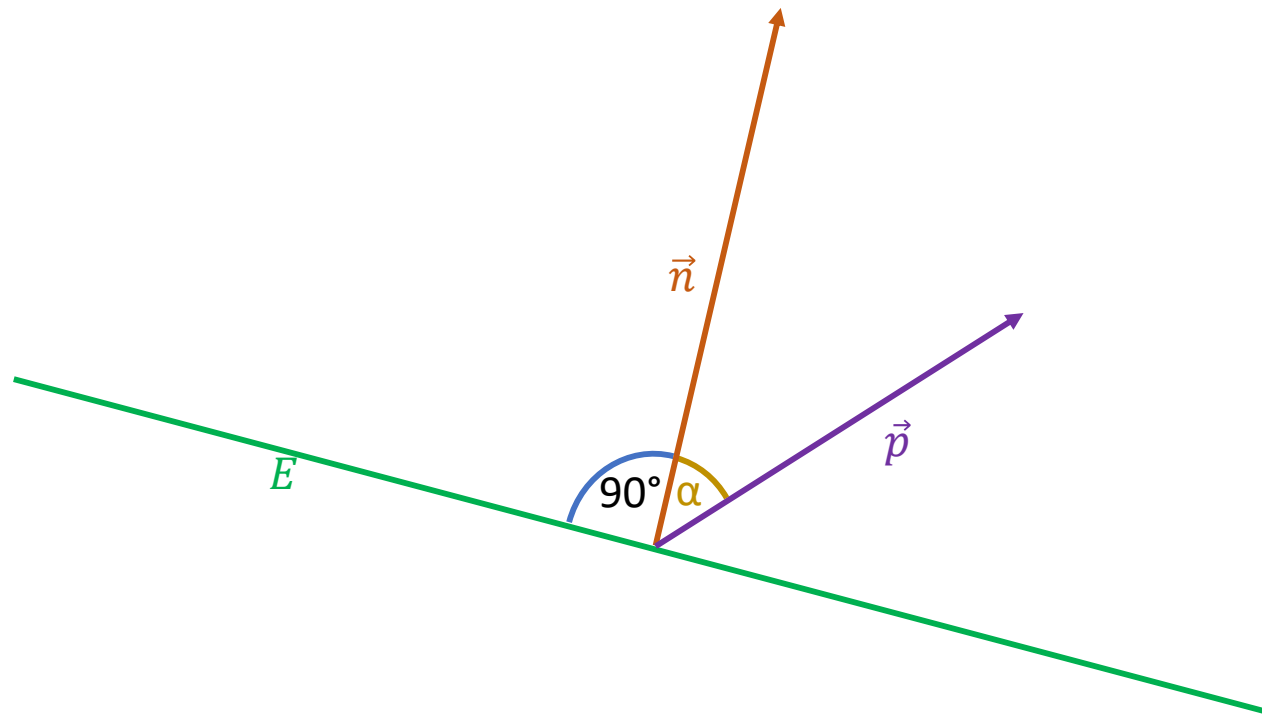
Zusammenhang: Skalarprodukt und Abstand

- Abstandsformel basiert auf Skalarprodukt $\vec{p}^T \vec{n}$ (+Konstante)
- Wichtige Eigenschaft des Skalarprodukts:

$$\vec{p}^T \vec{n} = |\vec{p}| |\vec{n}| \cos(\alpha)$$

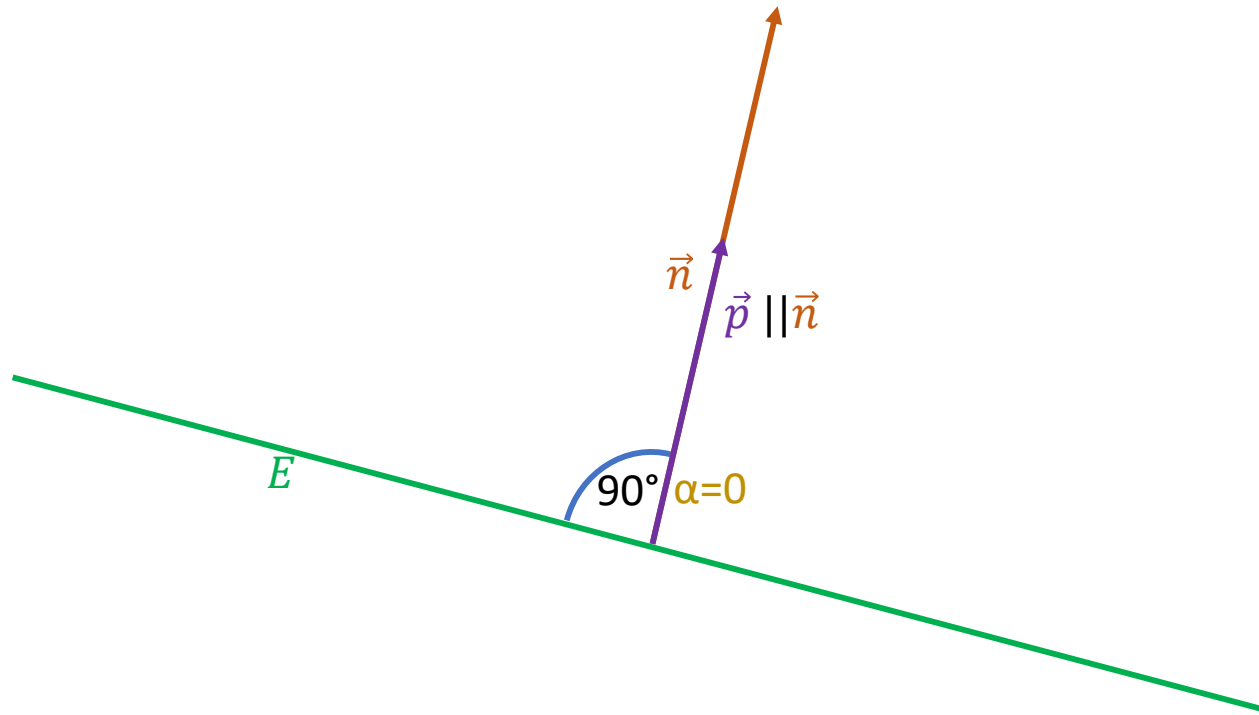
- Hier: α ist (innerer) Winkel zwischen $\vec{p}$ und $\vec{n}$

Geometrische Interpretation



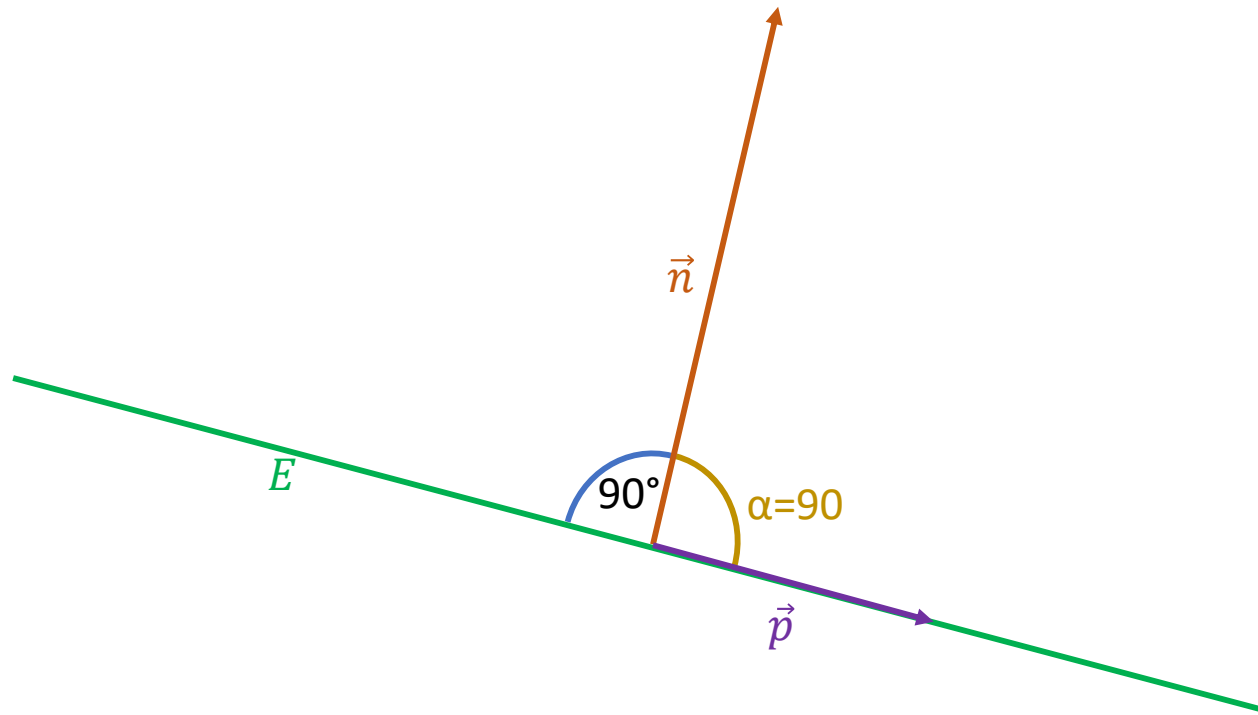
Annahme vorerst: E läuft durch Ursprung ($n_0 = 0$).

Geometrische Interpretation - parallel



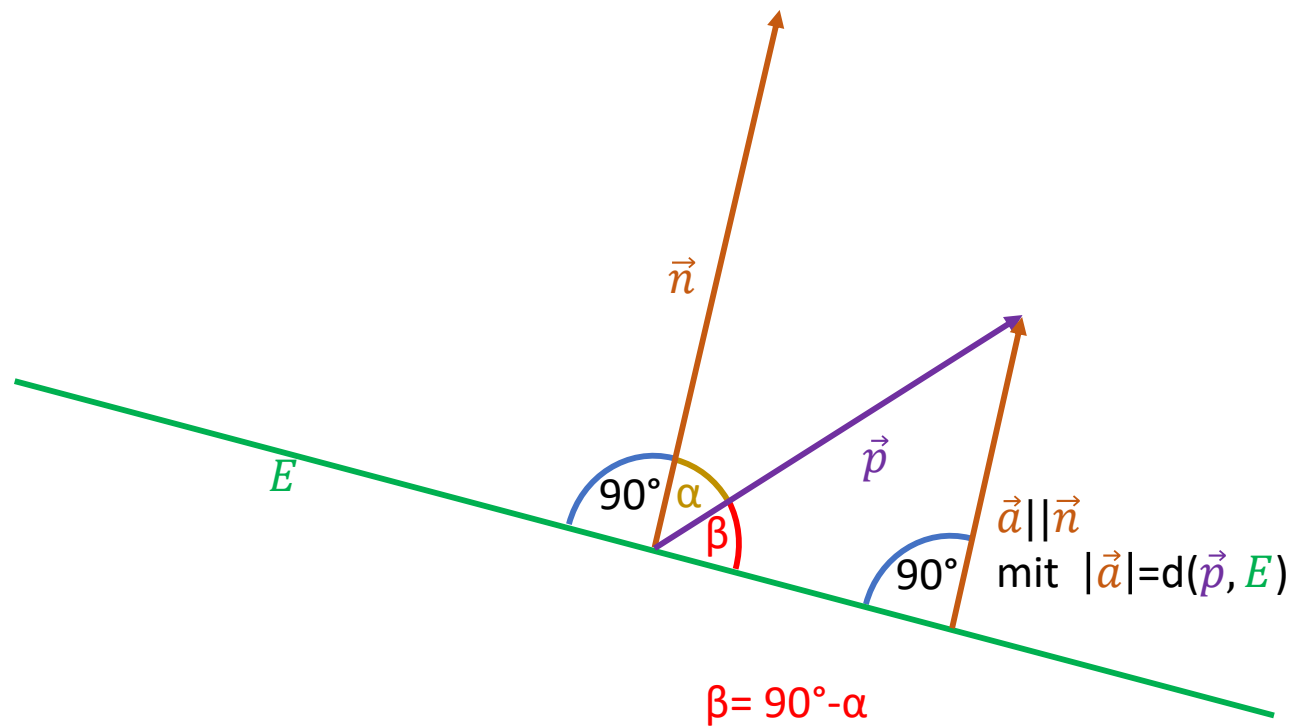
$\vec{p}$ ist parallel zu $\vec{n}$, also $d(\vec{p}, E) = |\vec{p}|$

Geometrische Interpretation - orthogonal



$\vec{p}$ ist orthogonal zu $\vec{n}$, also liegt $\vec{p}$ in E , somit $d(\vec{p}, E) = 0$

Geometrische Interpretation - beliebiger Winkel



Geometrische Interpretation

$$\sin(\beta) = |\vec{a}|/|\vec{p}|$$

$$\sin(90 - \alpha) = |\vec{a}|/|\vec{p}|$$

Regel zur Gleichheit von cos, sin:

$$\sin(90 - \alpha) = \cos(\alpha)$$

$$|\vec{a}| = \cos(\alpha) \cdot |\vec{p}|$$

Geometrische Interpretation

$$|\vec{a}| = \cos(\alpha) \cdot |\vec{p}|$$

Aber: wir kennen $\cos(\alpha)$ noch nicht.

$$\vec{p}^T \vec{n} = |\vec{p}| |\vec{n}| \cos(\alpha)$$

$$\frac{\vec{p}^T \vec{n}}{|\vec{p}| |\vec{n}|} = \cos(\alpha)$$

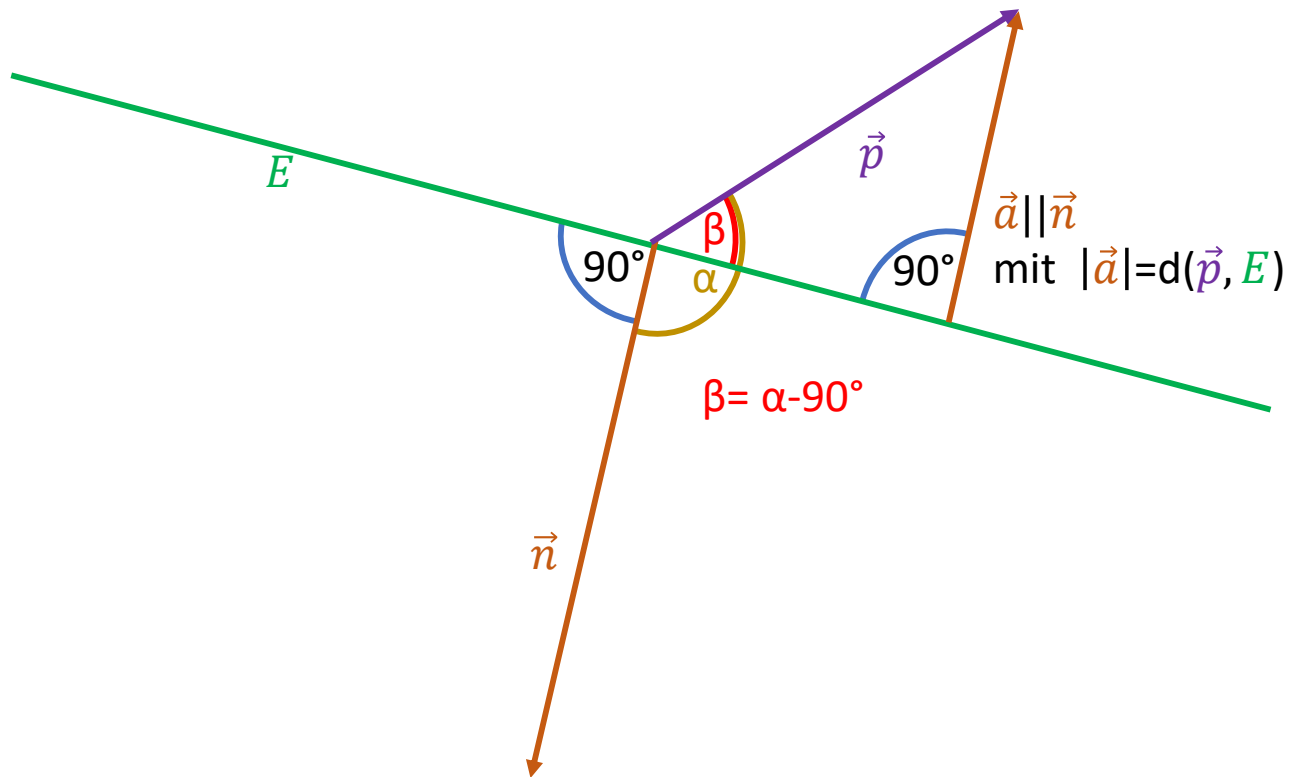
Einsetzen liefert:

$$|\vec{a}| = \frac{\vec{p}^T \vec{n}}{|\vec{p}| |\vec{n}|} \cdot |\vec{p}| = \frac{\vec{p}^T \vec{n}}{|\vec{n}|}$$

Bzw. wenn $\vec{n}$ normiert ist ($|\vec{n}| = 1$): $d(\vec{p}, E) = |\vec{a}| = \vec{p}^T \vec{n}$

Warum fehlen hier Betragsstriche aus der zuvor benutzten Formel?

Geometrische Interpretation - Richtungswechsel Normalenvektor



Änderung des Winkels führt zu Vorzeichenwechsel!

Geometrische Interpretation

- Wir wissen nun: Skalarprodukt kann als Abstand interpretiert werden
- Aber *nicht* Abstand zwischen den multiplizierten Vektoren!
- Abstand zwischen Vektor und einer Ebene
 - Ebene durch Ursprung
 - Normalenvektor ist *beliebiger* der beiden multiplizierten Vektoren
 - Skalarprodukt ändert sich nicht bei geeigneter Vertauschung $\vec{a}^T \vec{b}$ bzw. $\vec{b}^T \vec{a}$
 - Somit zwei Interpretationen möglich (jeweils mit unterschiedlicher Wahl des Normalenvektors)

Frage zur Geometrie

- Vorige Abbildungen nur Geometrie in 2D Raum
- Gilt das auch für 3D+?

Frage zur Geometrie

- Vorige Abbildungen nur Geometrie in 2D Raum
- Gilt das auch für 3D+?
 - Ja!
 - Die beiden betrachteten Vektoren spannen immer einen affinen Unterraum auf
 - Projektion der Vektoren in diesen Unterraum liefert die selben geometrischen Zusammenhänge die wir bereits für 2D kennen

Fazit

- Hyperebenengleichung: Skalarprodukt + Konstante
 - liefert **vorzeichenbehafteten Abstand** eines Punktes zu E
- Vorzeichen besagt, auf welcher Seite der Hyperebene ein Punkt bzw. Ortsvektor liegt
 - bzw. ob Normalenvektor und Ortsvektor in selbe Richtung zeigen

Anwendung: Margin Classifier

Classification

- Classification
 - Häufig betrachtete Zielsetzung für Machine Learning
 - Ziel: Trennung von Datenpunkten in 2 (oder mehr) Klassen (Gruppen von Datenpunkten)
 - Trainingsdaten: Klasse y_i jedes Datenpunktes $\vec{x}_i$ ist bekannt
 - Anwendung: Klasse y^* eines neuen Datenpunktes $\vec{x}^*$ ist unbekannt
- Methoden
 - Logistische Regression (LR)
 - Linear Discriminant Analysis (LDA)
 - Entscheidungsbäume
 - Neuronale Netze
 - ...

Classification

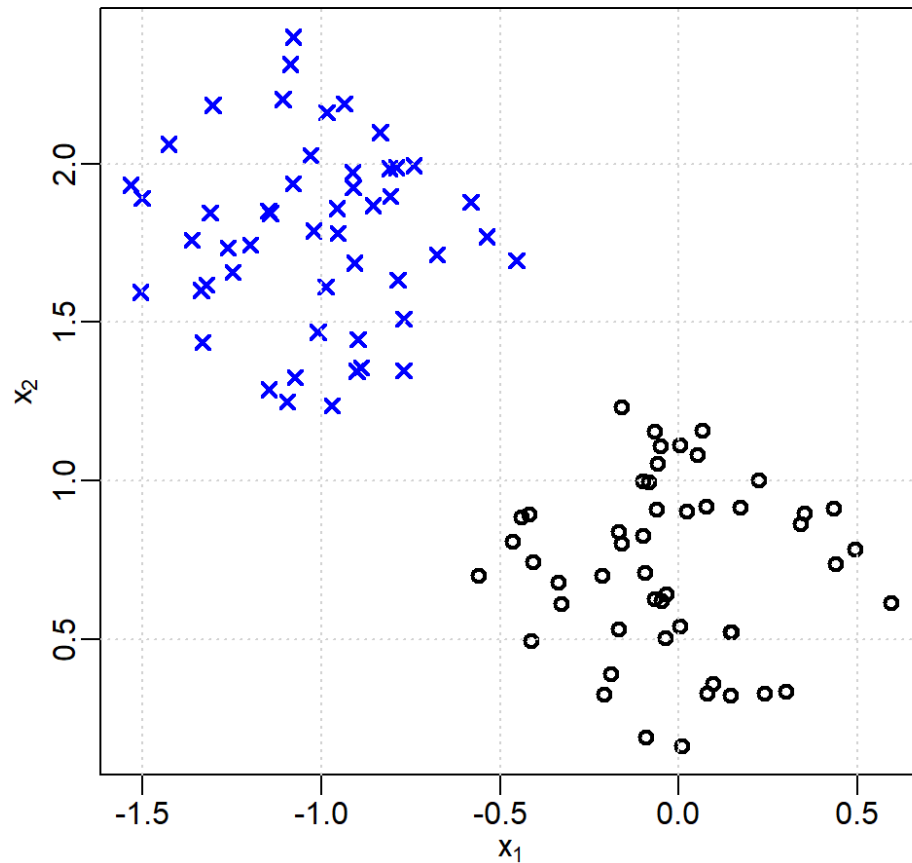
Wir betrachten Methoden, die explizit Hyperebenen betreffen:

- Hard Margin Classification
- Soft Margin Classification
 - a.k.a. Support Vector Classification
 - Vorstufe von Support Vector Machines

Einschränkung: Binary Classification, d.h., nur genau 2 Klassen

(Anmerkung: auch bei LR und LDA gibt es Bezüge zu Hyperebenen)

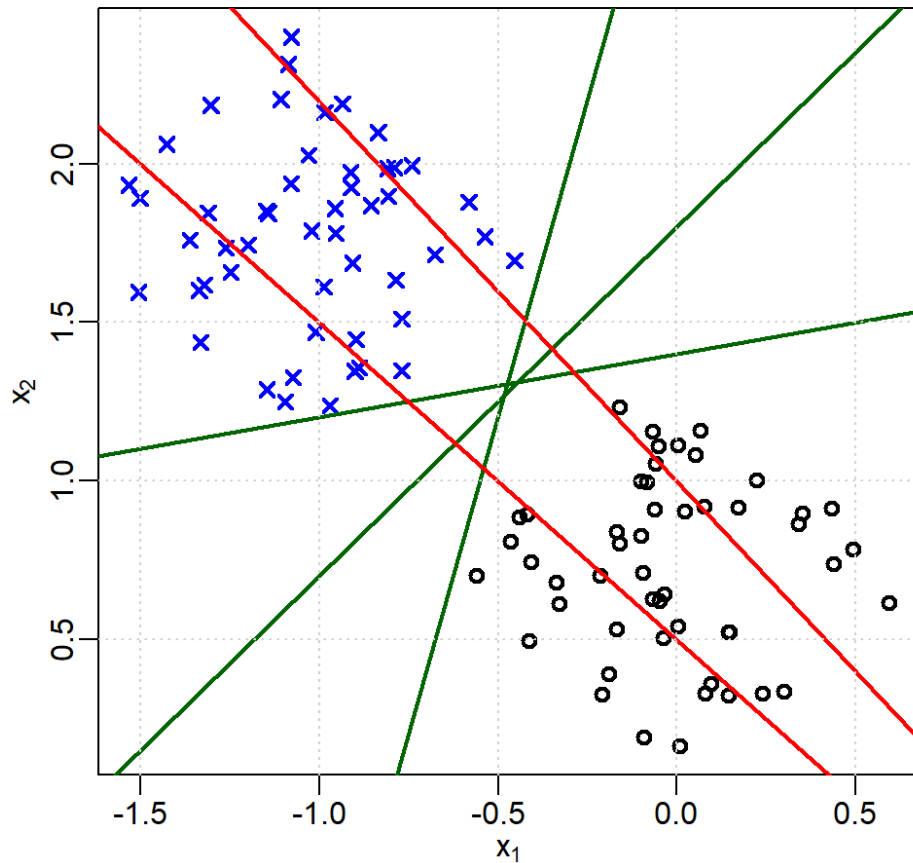
Binary Classification: Beispiel



Ziel: finde 'Regel' um beide Klassen ($\times$, o) gut zu Trennen

Frage: Wie würden Sie hier trennen?

Binary Classification: Trennende Hyperebenen



Wie können wir feststellen, ob eine Hyperebene die Datenpunkte exakt trennt (grün) oder nicht (rot)?

Bestimmen, ob trennende Hyperebene

Erinnerung:

- Koordinatengleichung der Hyperebene:

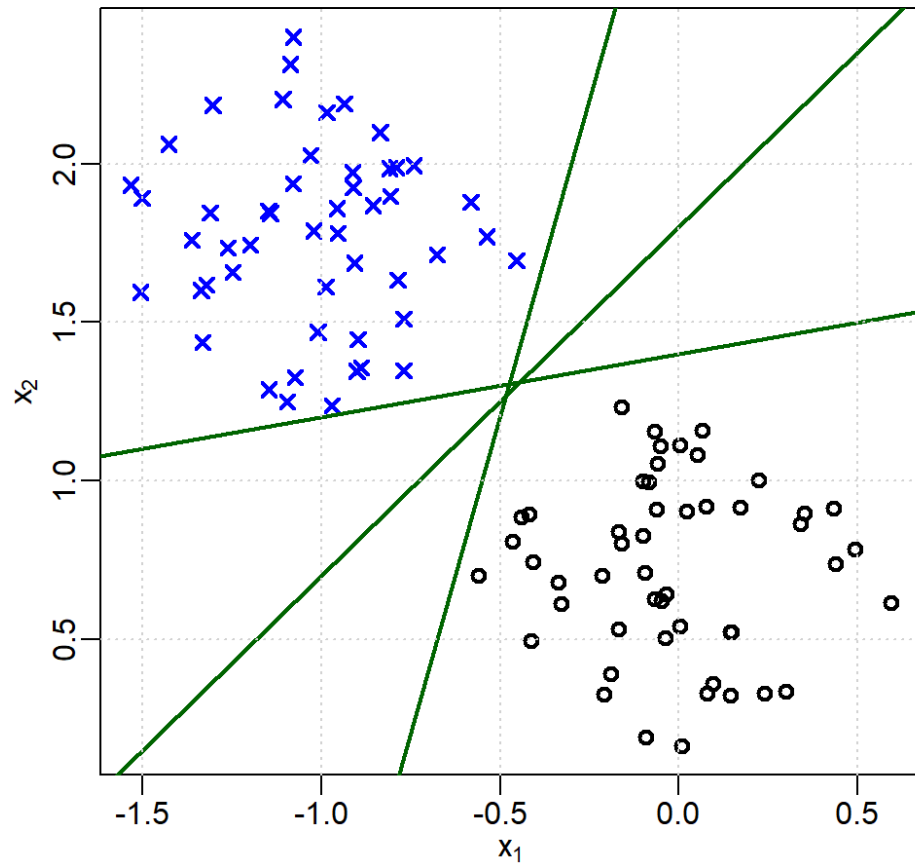
$$f(\vec{x}) = n_0 + \vec{n}^T \vec{x}$$

- $f(\vec{x}) > 0$ für Punkte auf einer Seite
- und $f(\vec{x}) < 0$ für Punkte auf anderer Seite

Trennende Hyperebene für Datenpunkte $\vec{x}_i, y_i$:

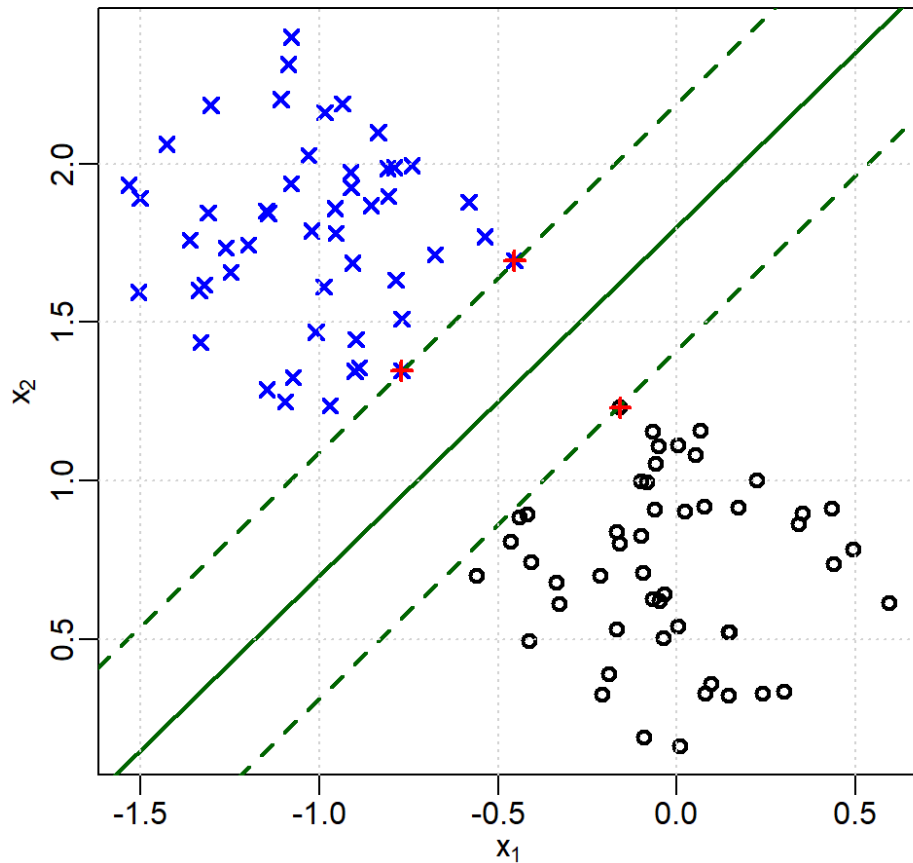
- Neukodierung der Klasse: $y_i = 1$ (Klasse $\times$) und $y_i = -1$ (Klasse o)
- Wenn $(y_i \cdot f(\vec{x}_i) > 0) \forall i$ ODER $(y_i \cdot f(\vec{x}_i) < 0) \forall i$
 - dann ist $f(\vec{x}) = 0$ trennende Hyperebene

Binary Classification: Trennende Hyperebenen



Aber welche der drei trennenden Hyperebenen ist die Beste?

Binary Classification: Hard margin



- Margin: minimaler Abstand zu Datenpunkten
- Hard margin classifier: trennende Hyperebene mit größtem Margin
- Supportvektoren: Punkte exakt auf den Margin-Grenzen (+)

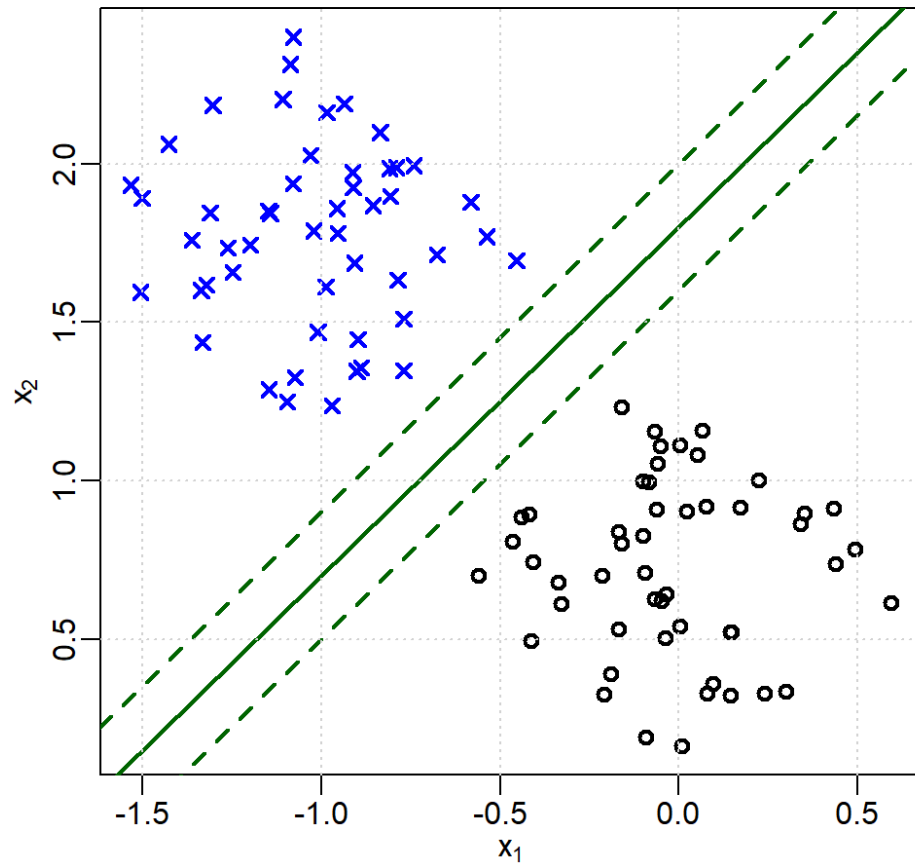
Warum "Supportvektoren"?

- Nur Supportvektoren "stützen" oder "bestimmen" die Hyperebene
 - Kleine Änderungen der Supportvektoren ändern die Lage der Ebene
 - Kleine Änderungen von nicht-Supportvektoren haben *keinen Einfluss* auf die Lage der Ebene

Ein paar intuitive Ideen

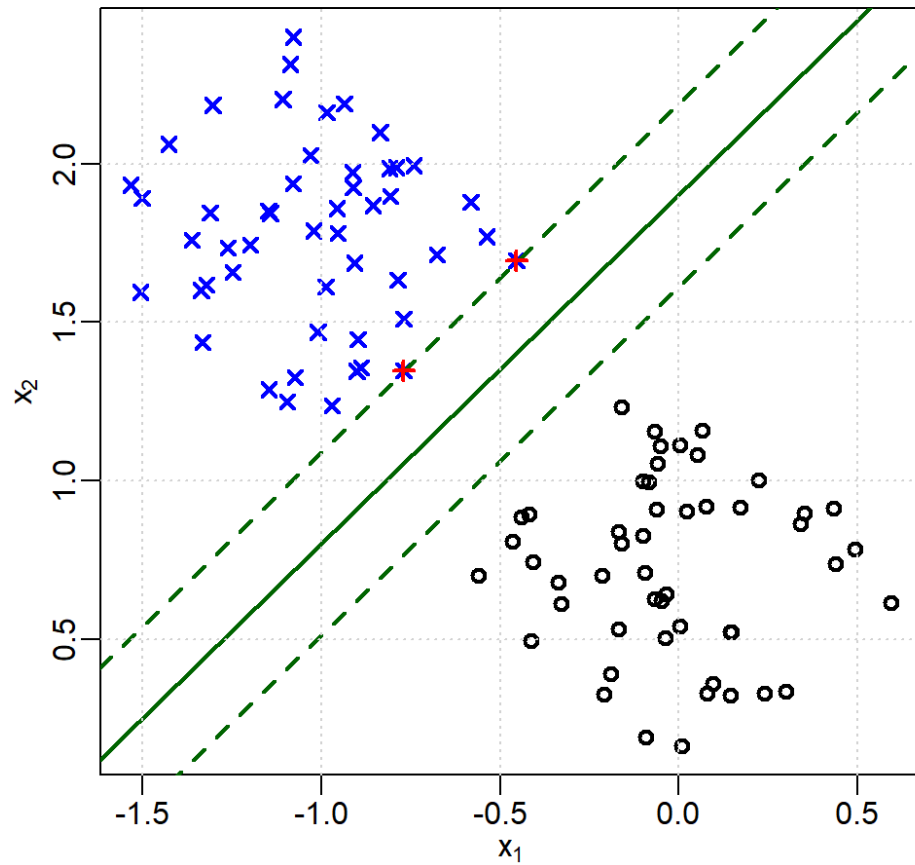
- Punkte innerhalb der Margin-Grenzen: verboten
- Wenn *keine* Supportvektoren vorhanden (d.h., kein Punkt liegt auf Grenze oder sogar dahinter)
 - Margin vergrößerbar, indem das Band um Gerade verbreitert wird
- Wenn in 2D Raum *zwei Supportvektoren* auf der *selben Seite* der Gerade liegen
 - Margin vergrößerbar, indem die Gerade von den Supportvektoren weggeschoben wird
- Wenn in 2D Raum *zwei Supportvektoren* auf der *unterschiedlichen Seiten* der Gerade liegen
 - Margin vielleicht vergrößerbar, indem die Gerade gedreht wird
- Bei maximalem Margin in 2D Raum
 - mindestens ein Supportvektoren auf einer Seite und mindestens ein Supportvektor auf der anderen

Ein paar intuitive Ideen



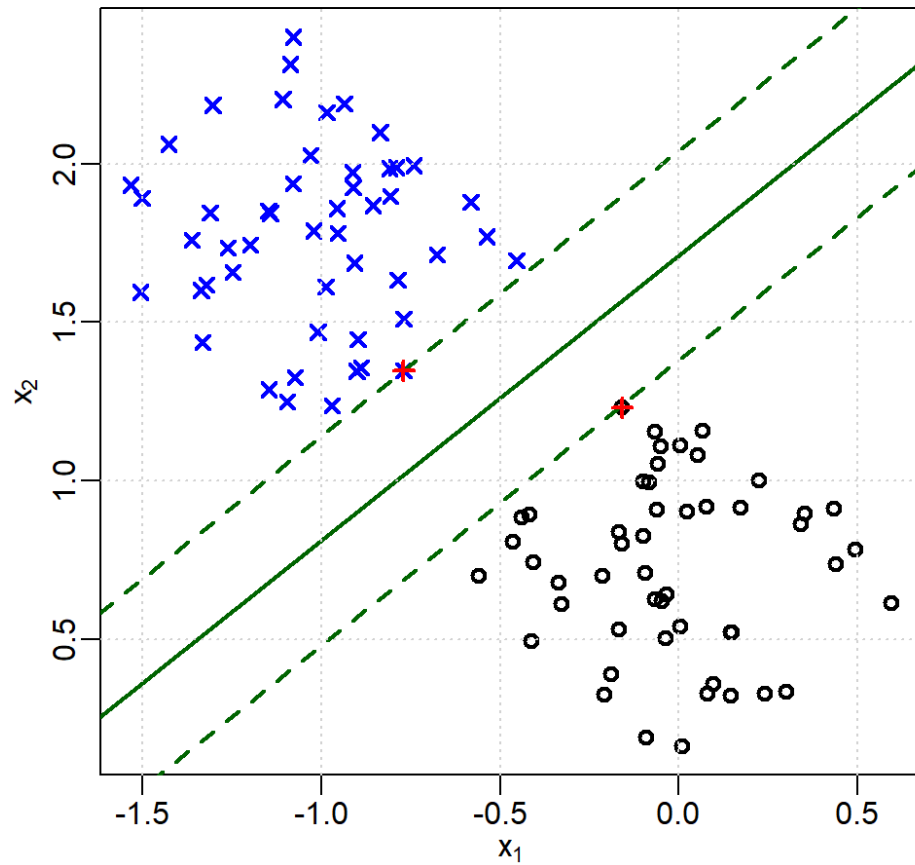
- Margin ist nicht maximal
- Band breiter machen

Ein paar intuitive Ideen



- Margin ist nicht maximal
- Gerade nach rechts unten schieben

Ein paar intuitive Ideen



- Margin ist nicht maximal
- Gerade drehen (um Mittelpunkt)

Bestimmen, ob Datenpunkte Margin einhalten

Trennende Hyperebene $f(x) = n_0 + \vec{n}^T \vec{x} = 0$ für Datenpunkte $\vec{x}_i, y_i$:

- Wenn $(y_i \cdot f(\vec{x}_i) > 0) \forall i$ ODER $(y_i \cdot f(\vec{x}_i) < 0) \forall i$
 - dann ist $f(\vec{x}) = 0$ trennende Hyperebene

Trennende Hyperebene für Datenpunkte $\vec{x}_i, y_i$, ohne Verletzung des Margins M :

- Wenn $(y_i \cdot f(\vec{x}_i) \geq M) \forall i$ ODER $(y_i \cdot f(\vec{x}_i) \leq -M) \forall i$
 - Voraussetzung: Normalenvektor $\vec{n}$ ist normiert!
 - Sonst steht auf linker Seite der Ungleichung keine Distanz

Maximierung des Margins

Optimierungsproblem mit Nebenbedingungen, für Daten mit p Spalten (Features) und m Zeilen (Datenpunkte $\vec{x}_i, y_i$)

$$\max_{n_0, n_1, \dots, n_p} M$$

Mit Nebenbedingungen $\sum_{j=1}^p n_j^2 = 1$

und $y_i \cdot (n_0 + \vec{n}^T \vec{x}_i) \geq M$

$$\forall i \in \{1, \dots, m\}$$

Anmerkung: Kann in quadratisches Problem umformuliert werden. Entsprechend häufig mit Algorithmen für quadratische Probleme gelöst.

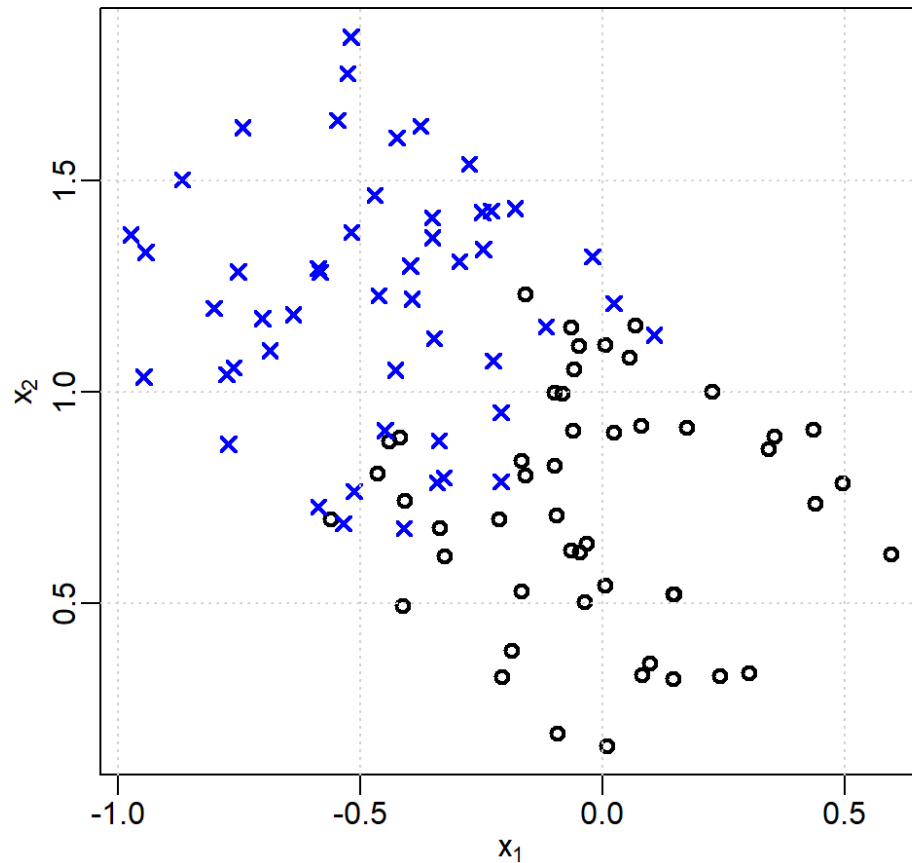
Zwischenfrage

Was bedeutet die Nebenbedingung

$$\sum_{j=1}^p n_j^2 = 1$$

und warum wird diese benötigt?

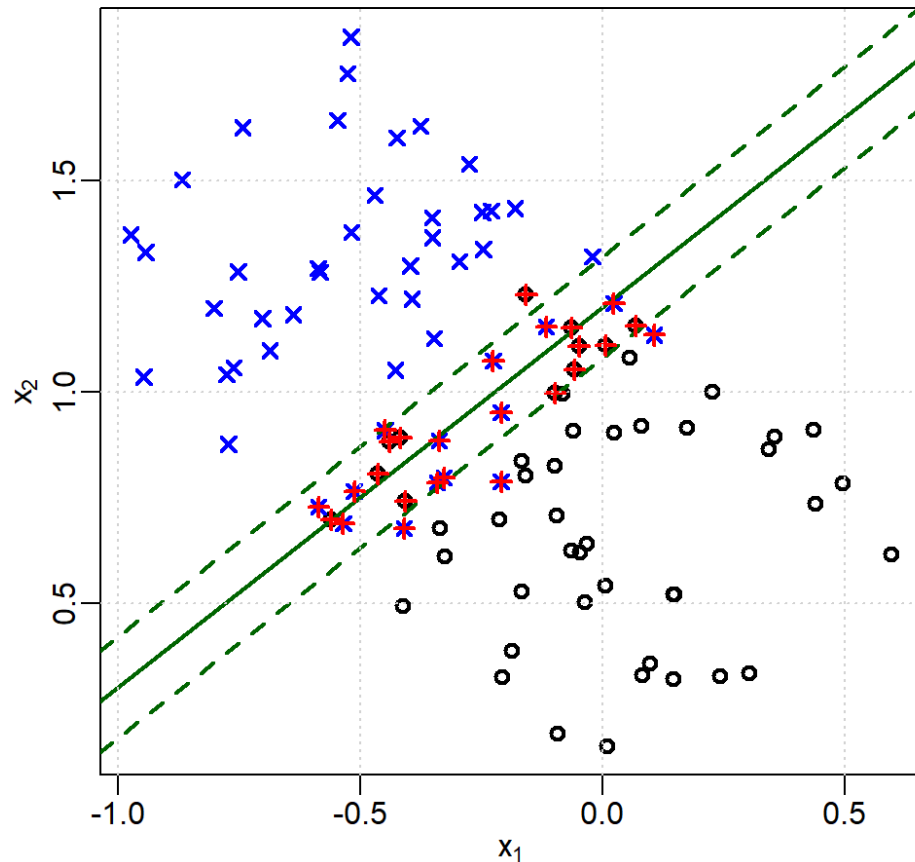
Grenzen des Hard Margins



- Problem: Trennende Hyperebene existiert nicht
 - Ist üblich, reale Daten sind häufig nicht linear trennbar (wenn $m \geq p$)
- Rückgriff auf nicht-lineare Methoden notwendig?

Soft Margin / Support vector classification

Support vector classifier



- Grundlegende Idee:
 - die Daten sind zwar nicht exakt linear trennbar
 - Aber vielleicht *ungefähr* linear trennbar
 - d.h., ein Großteil der Daten (aber nicht alle) wird durch Hyperebene getrennt
- Hier: Supportvektoren sind alle Punkte, die Margin verletzen
 - Exakt auf Margin Grenze
 - Innerhalb des Margins
 - Oder sogar auf falscher Seite der Hyperebene

Naiver support vector classifier

$$\max_{n_0, n_1, \dots, n_p; \epsilon_1, \dots, \epsilon_m} M \quad (1)$$

$$\text{Unter Nebenbedingung} \quad \sum_{j=1}^p n_j^2 = 1 \quad (2)$$

$$\text{und} \quad y_i \cdot (n_0 + \vec{n}^T \vec{x}_i) \geq (M - \epsilon_i) \quad (3)$$

$$\text{und} \quad \epsilon_i \geq 0 \quad (4)$$

$$\forall i \in \{1, \dots, m\} \quad (5)$$

Hier:

- ϵ_i vergibt einen Wert für jeden Datenpunkt $\vec{x}_i, y_i$
- Um diesen Wert darf der zugehörige Datenpunkt das Margin verletzen

Was halten Sie von dieser Definition? Funktioniert das?

Richtiger support vector classifier

Vorige Definition kann nicht funktionieren!

Es würden alle ϵ_i einfach beliebig groß gesetzt und so zu unbeschränkt großem Margin M führen.

Besser: Wir vergeben ein Budget C , das die Summe der ϵ_i nicht überschreiten darf!

$$\max_{n_0, n_1, \dots, n_p; \epsilon_1, \dots, \epsilon_m} M \quad (6)$$

$$\text{Unter Nebenbedingung} \quad \sum_{j=1}^p n_j^2 = 1 \quad (7)$$

$$\text{und} \quad y_i \cdot (n_0 + \vec{n}^T \vec{x}_i) \geq (M - \epsilon_i) \quad (8)$$

$$\text{und} \quad \epsilon_i \geq 0 \quad (9)$$

$$\forall i \in \{1, \dots, m\} \quad (10)$$

$$\text{und} \quad \sum_{i=1}^m \epsilon_i \leq C \quad (11)$$